Nama : Vika Elda Lestari
NPM : 4525210088

PERBAIKAN UTS PBO

Soal 4 — Inheritance dan Polymorphism

a. Output program

```
N:0812
S:0812
N:tu@kampus
SMS Kirim Halo ke 0812 x2
Kirim Halo ke tu@kampus x2
Total biaya 150
N:dosen@kampus
Email[Info] Kirim Rapat ke dosen@kampus
```

Penelusuran:

- new Sms("0812") menjalankan constructor Sms, yang langsung memanggil super(nomor). Karena itu N:0812 tercetak lebih dulu, baru S:0812.
- new Email("tu@kampus") hanya mencetak N:tu@kampus.
- Untuk Sms, n.kirim("Halo", 2) menghasilkan "SMS Kirim Halo ke 0812 x2" dan biaya() bernilai 150.
- Untuk Email, n.kirim("Halo", 2) menghasilkan "Kirim Halo ke tu@kampus x2" dan biaya() bernilai 0 (milik Notifikasi).
- Total biaya = 150 + 0 = 150.
- new Email("dosen@kampus") mencetak N:dosen@kampus. Lalu e.kirim("Rapat", "Info") memanggil kirim(String, String) milik Email, yang memanggil kirim("Rapat") milik Notifikasi.

b. Urutan method pada n.kirim("Halo", 2) saat n berisi object Sms

Urutannya: (1) Notifikasi.kirim(String, int), (2) Sms.kirim(String), (3) Notifikasi.kirim(String).

- Sms tidak memiliki kirim(String, int), tetapi method tersebut diwariskan dari Notifikasi (inheritance), sehingga method itu yang dijalankan.
- Di dalamnya ada pemanggilan kirim(pesan), yaitu this.kirim(pesan). Object sebenarnya adalah Sms, sehingga Java memilih versi override milik Sms saat runtime (dynamic binding / polymorphism).
- Sms.kirim(String) kemudian memanggil super.kirim(pesan), yang menjalankan versi milik Notifikasi.

c. Contoh overloading dan overriding

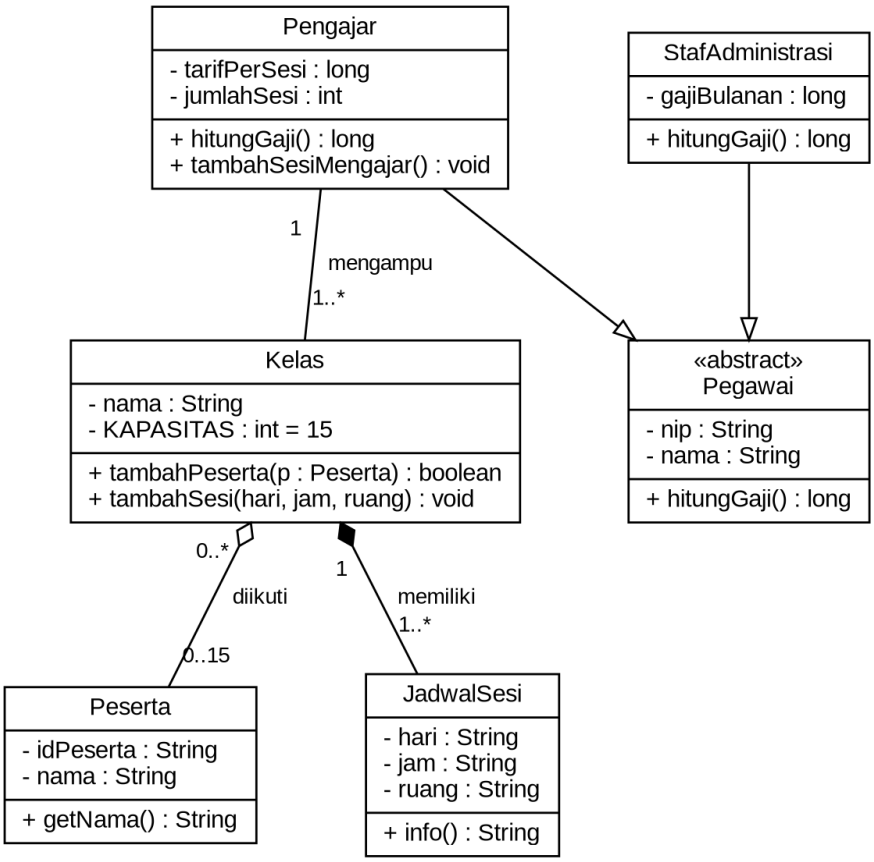
- Overloading: kirim(String) dan kirim(String, int) pada class Notifikasi (juga kirim(String, String) pada Email). Nama method sama, parameter berbeda.
- Overriding: kirim(String) dan biaya() pada Sms yang menimpa method Notifikasi. Signature sama persis.

Perbedaan cara Java memilih method:

- Overloading dipilih saat compile time (static binding), berdasarkan jumlah dan tipe parameter pada pemanggilan.
- Overriding dipilih saat runtime (dynamic binding), berdasarkan tipe object yang sebenarnya ditunjuk variabel, bukan tipe variabelnya.

Soal 5 — Analisis Kasus menjadi Class Diagram

a. Class diagram



Ringkasan relasi dan multiplicity:

Relasi	Jenis	Multiplicity
Pengajar dan StafAdministrasi dari Pegawai	Inheritance (generalisasi)	–
Pengajar — Kelas	Asosiasi	1 pengajar : 1..* kelas (satu kelas punya tepat 1 pengajar)
Kelas ◇— Peserta	Agregasi	Kelas 0..* : 0..15 Peserta (peserta dapat ikut banyak kelas)
Kelas ◆— JadwalSesi	Komposisi	1 Kelas : 1..* JadwalSesi

Catatan: Pegawai adalah class abstract dengan hitungGaji() abstract. Pengajar menghitungnya sebagai tarifPerSesi × jumlahSesi, sedangkan StafAdministrasi mengembalikan gajiBulanan.

b. Alasan pemilihan relasi

- Kelas–Pengajar = asosiasi. Narasi hanya menyatakan "setiap kelas diampu oleh satu pengajar, sedangkan seorang pengajar dapat mengampu beberapa kelas". Ini hubungan "mengampu" biasa, bukan hubungan bagian–keseluruhan. Pengajar adalah pegawai yang tetap ada walaupun kelasnya ditutup, dan kelas tidak "memiliki" pengajar.
- Kelas–Peserta = agregasi. Peserta adalah bagian dari kelas ("peserta mendaftar ke kelas, satu kelas berisi paling banyak 15 peserta"), tetapi kalimat "data peserta tetap disimpan walaupun kelasnya ditutup" menunjukkan peserta dapat hidup sendiri tanpa kelas. Peserta juga boleh mengikuti lebih dari satu kelas, sehingga bagian itu dipakai bersama. Ini ciri agregasi (bagian independen).
- Kelas–JadwalSesi = komposisi. Narasi menyebut jadwal sesi "dibuat bersama kelas tersebut dan ikut terhapus bila kelas ditutup". Jadwal bergantung penuh pada kelas, dibuat dan dihapus bersamanya, serta tidak dipakai kelas lain. Ini ciri komposisi (bagian tidak dapat hidup tanpa keseluruhan).

c. Kerangka kode class Kelas dan JadwalSesi

```
import java.util.ArrayList;
import java.util.List;

class JadwalSesi {
    private String hari;
    private String jam;
    private String ruang;

    // constructor tidak public: hanya dibuat oleh Kelas
    JadwalSesi(String hari, String jam, String ruang) {
        this.hari = hari;
        this.jam = jam;
        this.ruang = ruang;
    }

    public String info() {
        return hari + " " + jam + " @" + ruang;
    }
}

class Kelas {
    private static final int KAPASITAS = 15;

    private String nama;
    private Pengajar pengajar;                                  // asosiasi
    private List<Peserta> daftarPeserta = new ArrayList<>();    // agregasi
    private List<JadwalSesi> daftarSesi = new ArrayList<>();    // komposisi

    public Kelas(String nama, Pengajar pengajar) {
        this.nama = nama;
        this.pengajar = pengajar;
    }

    // AGREGASI: Peserta dibuat di luar, Kelas terima referensinya
    public boolean tambahPeserta(Peserta p) {
        if (p != null && daftarPeserta.size() < KAPASITAS) {
            daftarPeserta.add(p);
            return true;
        }
        return false;
    }

    // KOMPOSISI: JadwalSesi dibuat (new) di dalam Kelas
```

```
    public void tambahSesi(String hari, String jam, String ruang) {
        daftarSesi.add(new JadwalSesi(hari, jam, ruang));
    }
}
```

Perbedaan agregasi dan komposisi dalam kode:

- tambahPeserta(Peserta p) menerima object yang sudah ada dari luar. Object Peserta tetap hidup walaupun Kelas dibuang, dan dapat dimasukkan ke kelas lain.
- tambahSesi(hari, jam, ruang) menerima data mentah lalu membuat new JadwalSesi(...) di dalam Kelas. Jadwal hanya dirujuk oleh kelas itu, sehingga ikut hilang bersama kelasnya.